The National University of Lesotho

Department of Mathematics and Computer Science

Faculty of Science and Technology

CS4430: Distributed Database Systems

CENTRALIZED DATABASE DESIGN

Coordinator: Mr. Khobatha Setetemela

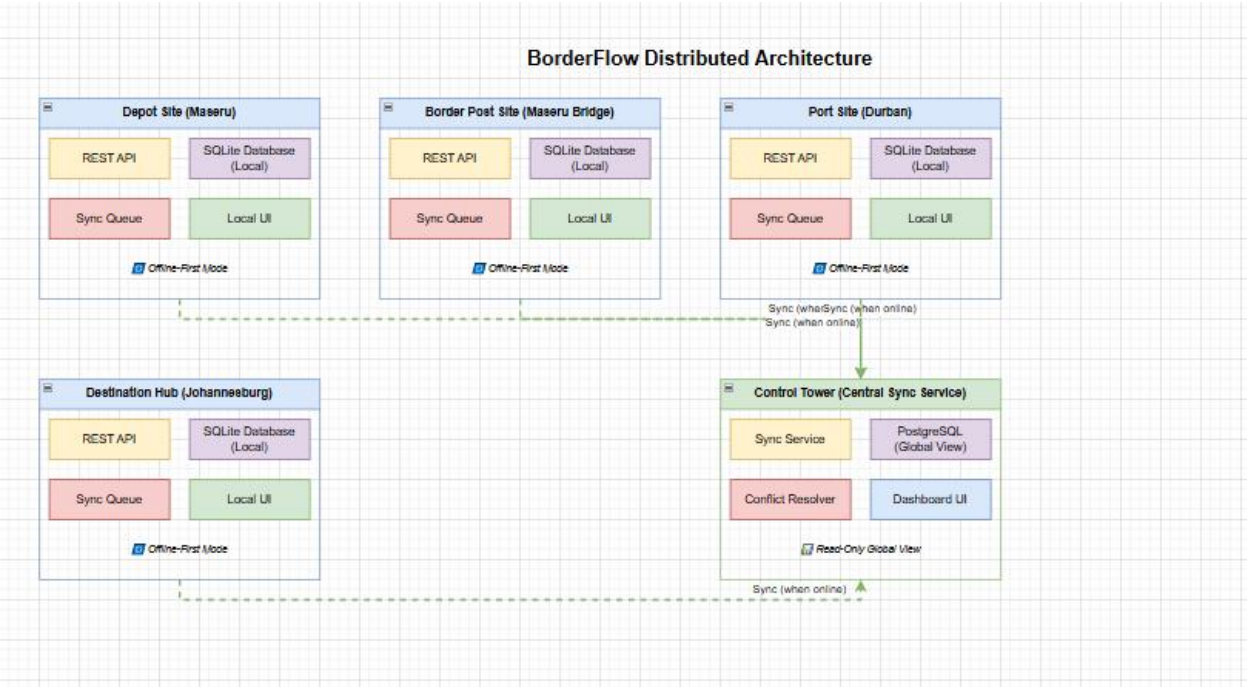
Due: April 2026

Team Members

Student Number	Name
200900706	Moleleki H.G.
202201934	Nkolokoane L.N.
202201176	Mohlapisi B.I.
202201173	Ramochele M.B.
202200990	Motaung M.
202201992	Makoetje K.G.M.
202201955	Leiee L.J.
202201074	Ramainoane M.A.R.

BorderFlow: Distributed Database Design

Architecture



2. Architecture Rationale

2.1 Design Decisions and Justification

Multi-Master Replication: All sites accept writes independently. Prioritizes availability over strong consistency (AP from CAP theorem). Essential for offline-first operation where network partitions are expected.

Control Tower as Central Sync Hub: Simplifies discovery and reduces peer-to-peer complexity. Each site only needs to know one endpoint. Avoids gossip protocol overhead and split-brain scenarios.

SQLite per Site: Lightweight, zero-configuration, embedded database. Perfect for edge deployments. No separate database server process needed. Survives process restarts.

PostgreSQL at Control Tower: Robust, mature relational database for global view. Supports ACID transactions for conflict resolution. Handles concurrent sync requests reliably.

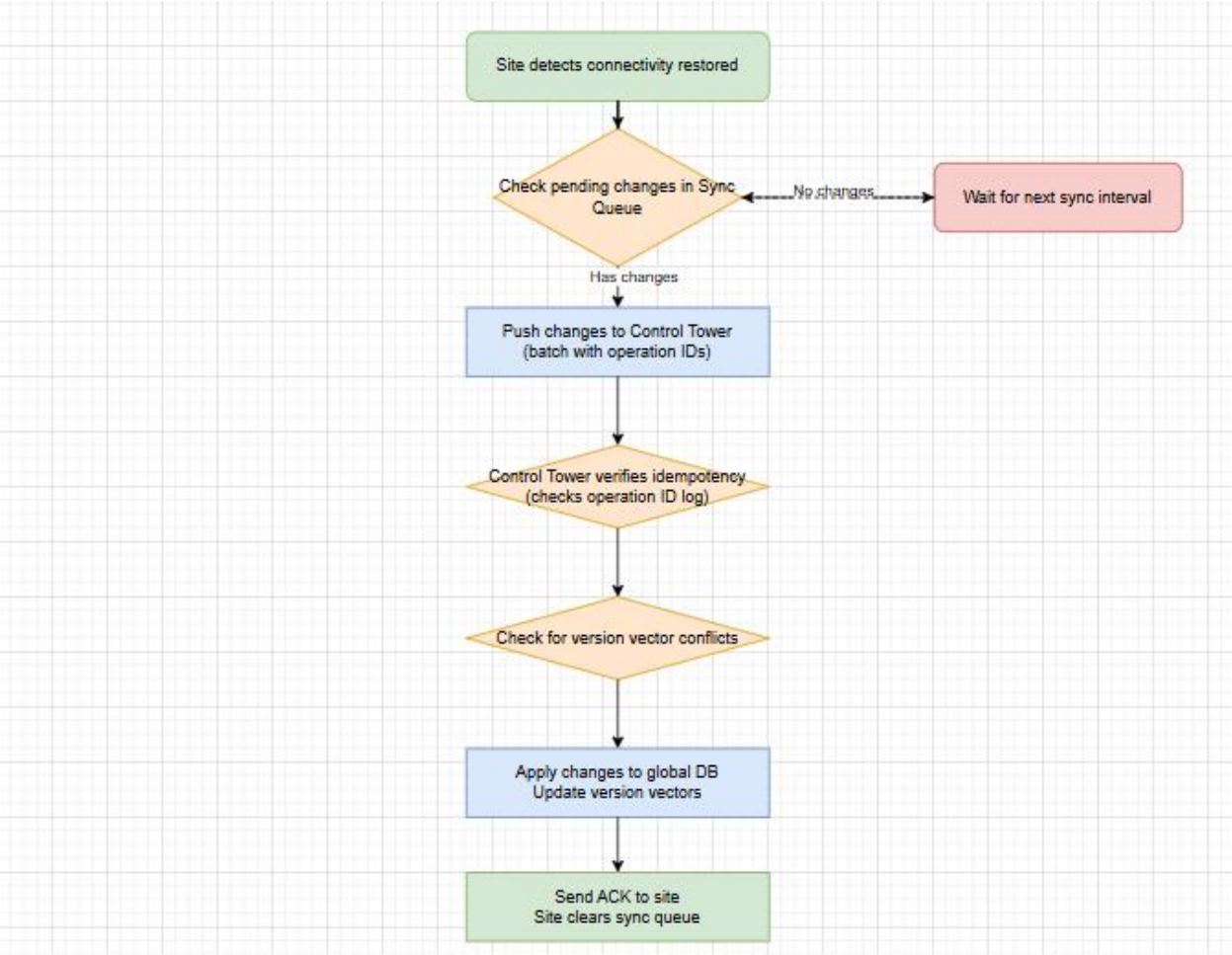
Push-Based Synchronization: Sites proactively push changes when connectivity is restored. Simpler than pull-based with polling. Guarantees changes eventually reach the hub.

REST APIs: Platform-agnostic, simple to implement, easy to debug. Standard HTTP with JSON payloads. Works across firewalls and network boundaries.

2.2 Topology Comparison

Topology	Pros	Cons	BorderFlow Choice
Peer-to-Peer	No central dependency	Complex discovery, broadcast overhead	no
Central Hub	Simple, single sync point	Hub becomes single point of failure	yes(with recovery)
Hybrid	Flexible	Complex to implement	no

3. Synchronization Strategy



4. Conflict Resolution Strategy

4.1 Conflict Types and Resolution

Conflict Type	Example	Resolution Strategy
Concurrent Updates	Depot and Border both update same container status offline	Version vector detection → Last Write Wins (timestamp + site ID tiebreak
Duplicate Operations	Same milestone reported twice due to retry	Idempotency keys → ignore duplic

Conflict Type	Example	Resolution Strategy
Missing Dependencies	Milestone arrives before parent Trip	Defer processing, queue for later sync
Semantic Conflicts	Depot says "DELIVERED", Port says "IN_TRANSIT"	Flag for manual resolution in Contr Tower
Assignment Conflicts	Same driver assigned to two trips offline	Reject second assignment, notify both sites